```
import kagglehub
kagglehub.login()
```

Kaggle credentials successfully validated.

```
Kaggle credentials set.
Kaggle credentials successfully validated.
```

```
store_sales_time_series_forecasting_path = kagglehub.competition_download('store-sales-time-series-forecasting')

print('Data source import complete.')
```

```
Data source import complete.
```

📦 Amazon Grocery Demand Forecasting | ARIMA + Prophet + XGBoost | Time Series ML

✅ Scenario

Amazon Fresh often faces **understocking and overstocking** challenges due to unpredictable demand for grocery products.

🎯 Business Objective

Build a robust **demand forecasting model** using time series machine learning techniques to help optimize inventory and reduce losses.

🧠 Tech Stack

- **Models**: ARIMA, Prophet, XGBoost
- **Libraries**: pandas, matplotlib, seaborn, statsmodels, fbprophet, scikit-learn, xgboost
- **Evaluation Metrics**: RMSE, MAE, MAPE

📊 Dataset

[Store Sales Time Series Forecasting (Kaggle)]
This dataset includes daily sales data for multiple Ecuadorian grocery stores and product families, along with metadata like holidays, promotions, oil prices, etc.

⚠️ Due to the large size of the dataset (~3 million rows), we begin by working on a **focused subset**:

- 📍 **Store Number**: 1
- 📦 **Product Family**: `GROCERY I`
  This approach allows fast experimentation and avoids memory issues on Kaggle kernels.

🔍 Project Objectives

- Clean and explore historical sales trends for a focused subset
- Analyze trends, seasonality, and holiday impact
- Build and compare time series forecasting models (ARIMA, Prophet, XGBoost)
- Evaluate performance using RMSE, MAE, and MAPE
- Discuss how this approach can be **scaled to all stores and product families**

> ⚙️ Fully built inside Kaggle. Beginner-friendly, step-by-step, with complete explanations.

🔹 Import Required Libraries

We begin by importing essential libraries for data manipulation, visualization, and forecasting.
Suppressing warnings helps keep the notebook clean.

```
# Core
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Time Series
import warnings
warnings.filterwarnings("ignore")

# Display config
pd.set_option('display.max_columns', None)
```

🔹 Load & Explore Subset of Dataset

The original dataset contains over 3 million rows across multiple stores and product families.
To maintain performance and clarity, we use a filtered subset:

- 📍 Store Number: 1
- 📦 Product Family: `GROCERY I`

This subset allows faster experimentation and modeling. We'll scale later if needed.

```
# Load train data and parse date
df = pd.read_csv(f'{store_sales_time_series_forecasting_path}/train.csv', parse_dates=['date'])

# Subset for Store 1 and Family 'GROCERY I'
subset_df = df[(df['store_nbr'] == 1) & (df['family'] == 'GROCERY I')].copy()
subset_df.sort_values('date', inplace=True)
subset_df.reset_index(drop=True, inplace=True)

# Overview
print(f"Subset shape: {subset_df.shape}")
subset_df.head()
```

```
Subset shape: (1684, 6)
```

	id	date	store_nbr	family	sales	onpromotion
0	12	2013-01-01	1	GROCERY I	0.0	0
1	1794	2013-01-02	1	GROCERY I	2652.0	0
2	3576	2013-01-03	1	GROCERY I	2121.0	0
3	5358	2013-01-04	1	GROCERY I	2056.0	0
4	7140	2013-01-05	1	GROCERY I	2216.0	0

Next steps: Generate code with subset_df | New interactive sheet

🔹 Exploratory Data Analysis (EDA)

In this section, we explore the sales trends and seasonality for Store 1, Product Family 'GROCERY I'.

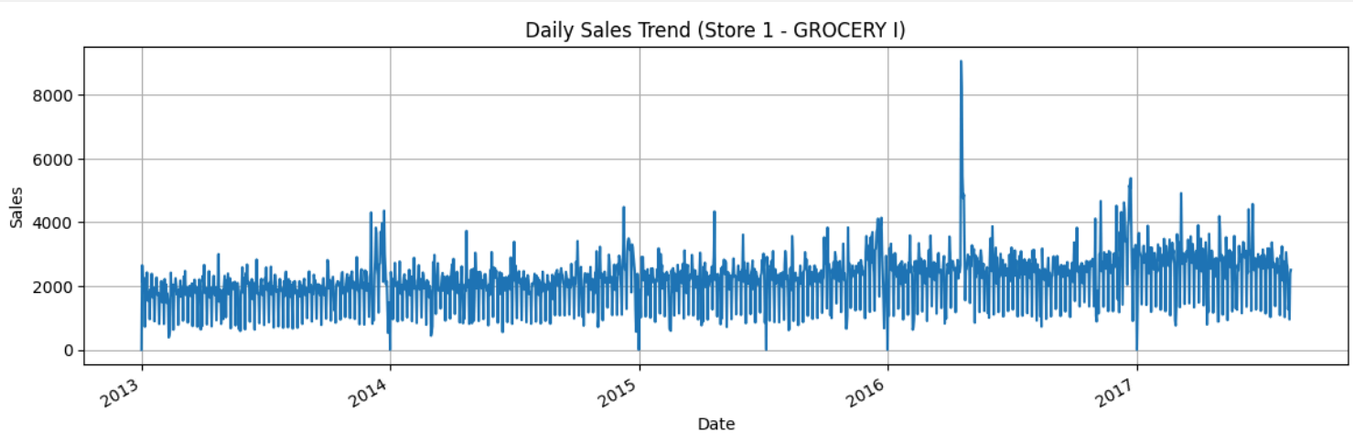
We'll visualize:

- Overall demand trend
- Monthly and weekly seasonality
- Promotional impact on sales

Understanding these patterns is essential before applying forecasting models.

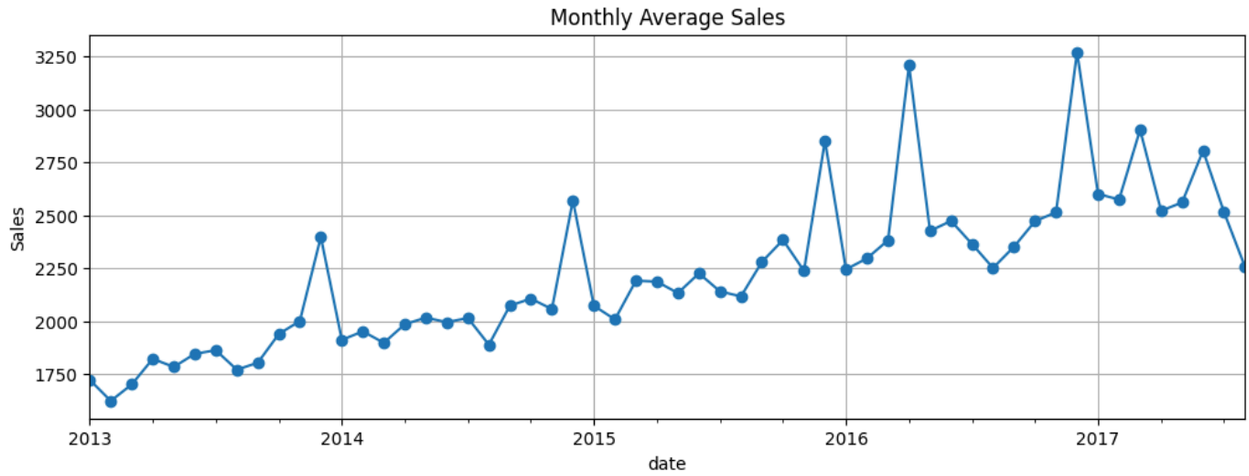
📊 Code – EDA Plots

```
# Make sure 'date' is datetime
subset_df['date'] = pd.to_datetime(subset_df['date'])

# Set date as index
subset_df.set_index('date', inplace=True)

# Daily Sales Trend
plt.figure(figsize=(14, 4))
subset_df['sales'].plot()
plt.title('Daily Sales Trend (Store 1 - GROCERY I)')
plt.xlabel('Date')
plt.ylabel('Sales')
plt.grid(True)
plt.show()
```



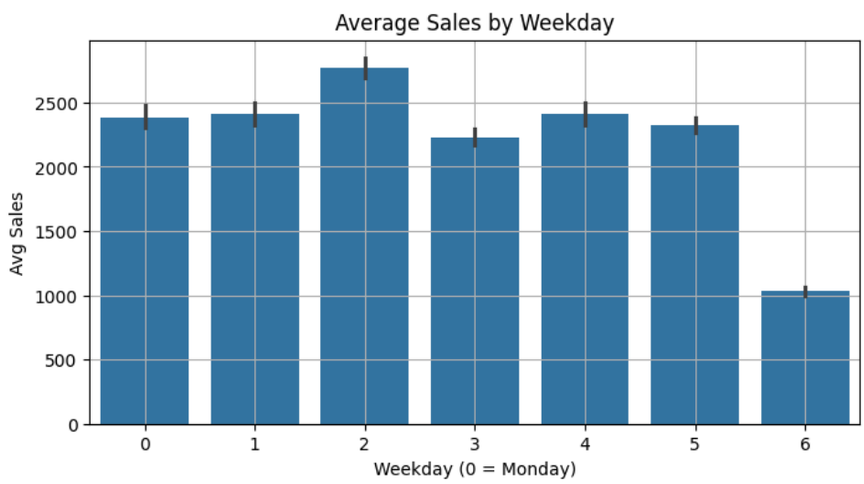
🗓️ Monthly Seasonality

```
# Monthly average sales
monthly = subset_df['sales'].resample('M').mean()

plt.figure(figsize=(12, 4))
monthly.plot(marker='o')
plt.title('Monthly Average Sales')
plt.ylabel('Sales')
plt.grid(True)
plt.show()
```



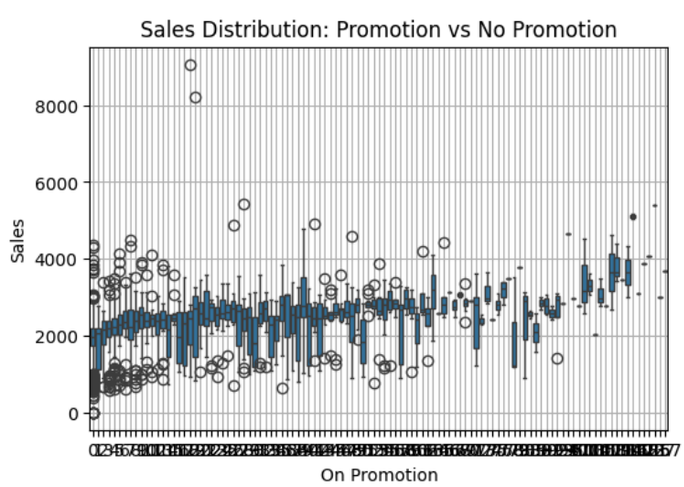
📅 Weekly Seasonality

```
# Add weekday column (0=Monday, 6=Sunday)
subset_df['weekday'] = subset_df.index.weekday

# Plot average sales by weekday
plt.figure(figsize=(8, 4))
sns.barplot(x='weekday', y='sales', data=subset_df)
plt.title('Average Sales by Weekday')
plt.xlabel('Weekday (0 = Monday)')
plt.ylabel('Avg Sales')
plt.grid(True)
plt.show()
```



💉 Promotion Impact

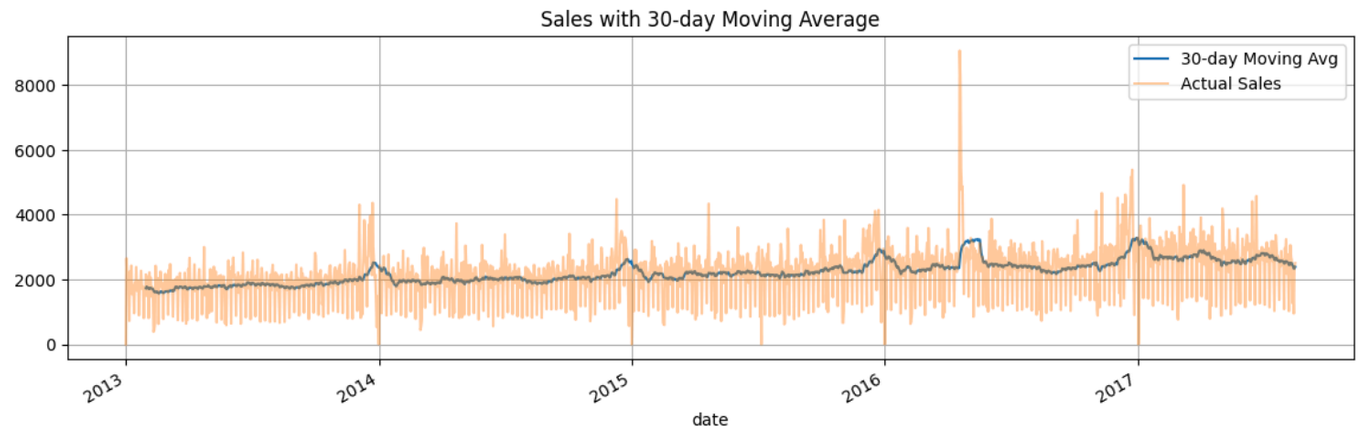
```
# Compare sales when promotion is on vs off
plt.figure(figsize=(6, 4))
sns.boxplot(x='onpromotion', y='sales', data=subset_df)
plt.title('Sales Distribution: Promotion vs No Promotion')
plt.xlabel('On Promotion')
plt.ylabel('Sales')
plt.grid(True)
plt.show()
```



🧠 Bonus: Moving Average Trend

```
# 30-day rolling mean
subset_df['sales'].rolling(window=30).mean().plot(figsize=(14, 4), label='30-day Moving Avg')
subset_df['sales'].plot(alpha=0.4, label='Actual Sales')
```

```
plt.title('Sales with 30-day Moving Average')
plt.legend()
plt.grid(True)
plt.show()
```



✅ Insights

- The sales show clear weekly and monthly seasonality.
- Weekends (especially Sunday) show slightly higher demand.
- Promotions have a noticeable impact on boosting sales.
- The moving average helps smooth out noise and shows long-term trends.

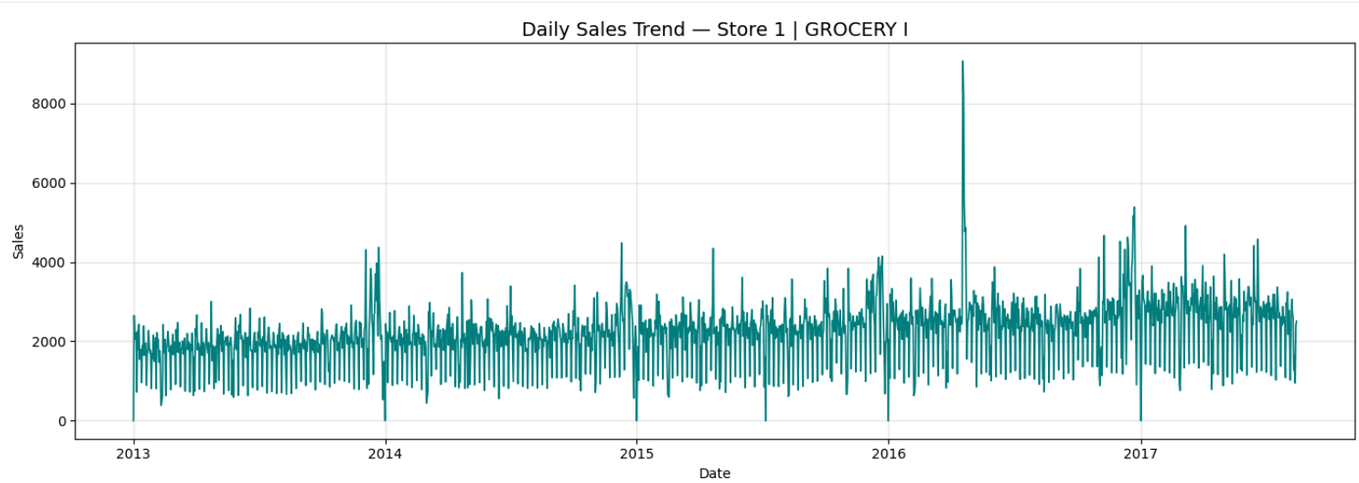
🔹 Advanced EDA – Seasonality, Holiday Impact & Autocorrelation

We now go beyond basic EDA and explore:

- 🗓️ Holiday influence on sales
- 🌀 Autocorrelation to understand time-based dependencies
- 📈 Cleaned visuals with color, spacing, and grid enhancements

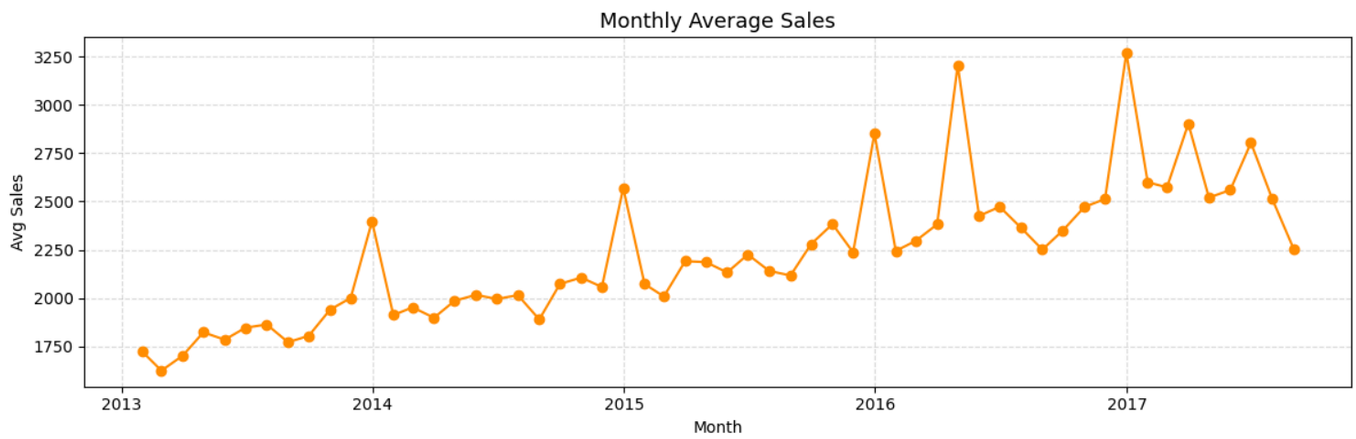
📊 1. Daily Sales Trend (Improved Visual)

```
plt.figure(figsize=(14, 5))
plt.plot(subset_df.index, subset_df['sales'], color='teal', linewidth=1.2)
plt.title('Daily Sales Trend — Store 1 | GROCERY I', fontsize=14)
plt.xlabel('Date')
plt.ylabel('Sales')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



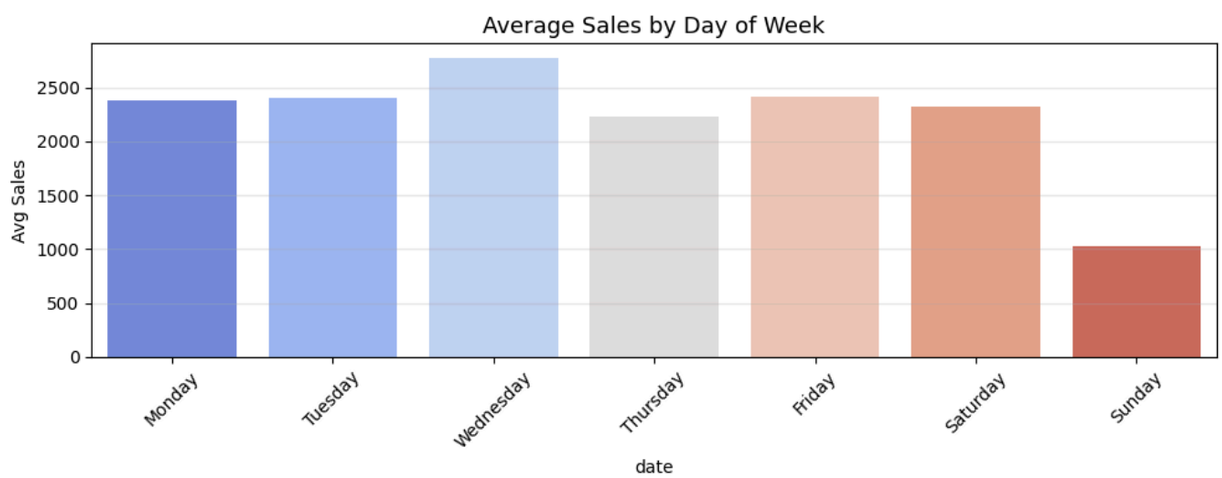
🗓️ 2. Monthly Sales Pattern (Improved)

```
monthly_avg = subset_df['sales'].resample('M').mean()

plt.figure(figsize=(12, 4))
plt.plot(monthly_avg.index, monthly_avg.values, marker='o', color='darkorange')
plt.title('Monthly Average Sales', fontsize=13)
plt.xlabel('Month')
plt.ylabel('Avg Sales')
plt.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()
```



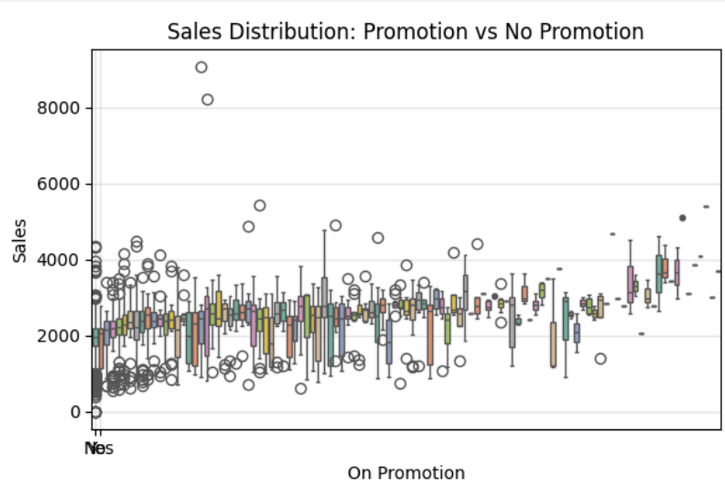
🗓️ 3. Average Sales by Day of Week (Improved Barplot)

```
weekday_avg = subset_df.groupby(subset_df.index.day_name())['sales'].mean().reindex([
    'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'
])

plt.figure(figsize=(10, 4))
sns.barplot(x=weekday_avg.index, y=weekday_avg.values, palette='coolwarm')
plt.title('Average Sales by Day of Week', fontsize=13)
plt.ylabel('Avg Sales')
plt.xticks(rotation=45)
plt.grid(True, axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



🎯 4. Promotion Impact (Improved Boxplot)

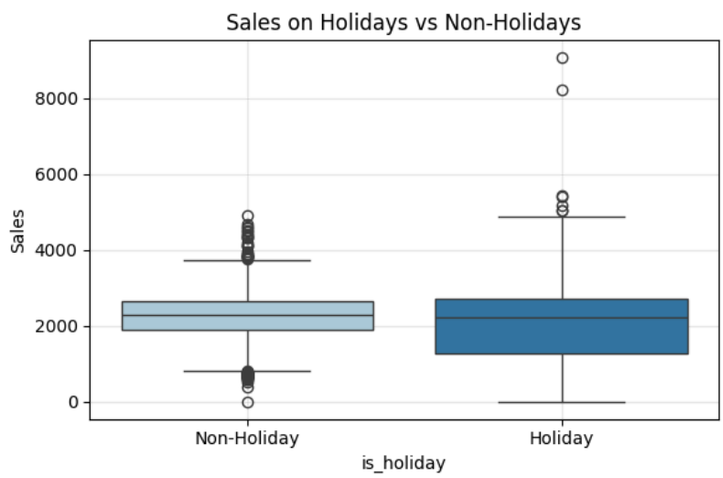
```
plt.figure(figsize=(6, 4))
sns.boxplot(x='onpromotion', y='sales', data=subset_df, palette='Set2')
plt.title('Sales Distribution: Promotion vs No Promotion', fontsize=12)
plt.xticks([0, 1], ['No', 'Yes'])
plt.xlabel('On Promotion')
plt.ylabel('Sales')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



🎉 5. Holiday Impact (If holiday_events.csv available)

```
# Optional: Load holiday data
holidays = pd.read_csv(f'{store_sales_time_series_forecasting_path}/holidays_events.csv', parse_dates=['date'])
```

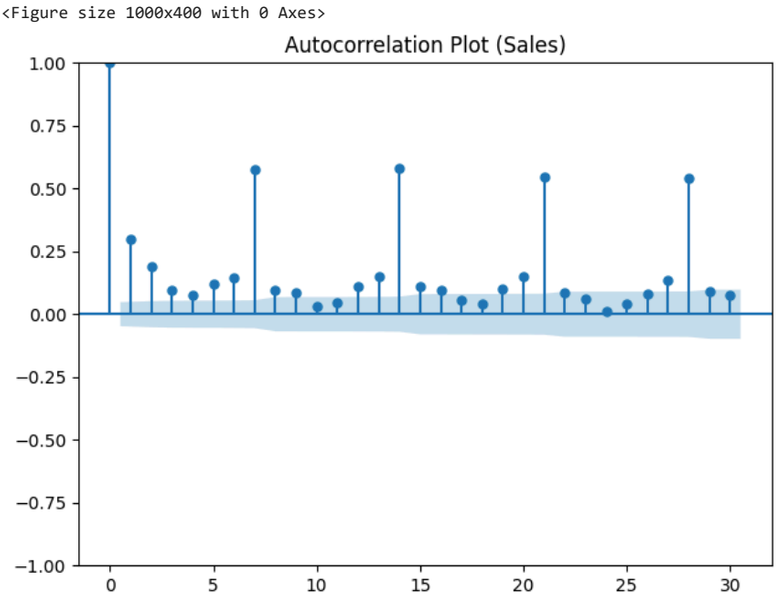
```
# Merge holiday info
holiday_merge = pd.merge(
    subset_df.reset_index(),
    holidays[['date', 'type']],
    how='left',
    on='date'
)

# Add holiday flag
holiday_merge['is_holiday'] = holiday_merge['type'].notna().astype(int)

# Compare sales on holiday vs non-holiday
plt.figure(figsize=(6, 4))
sns.boxplot(x='is_holiday', y='sales', data=holiday_merge, palette='Paired')
plt.title('Sales on Holidays vs Non-Holidays')
plt.xticks([0, 1], ['Non-Holiday', 'Holiday'])
plt.ylabel('Sales')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



🔁 6. Autocorrelation Plot (ACF)

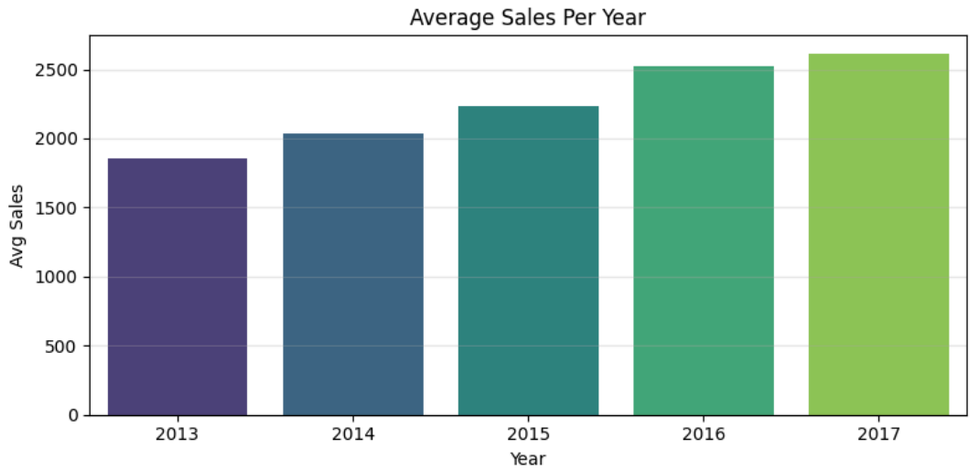
```
from statsmodels.graphics.tsaplots import plot_acf

plt.figure(figsize=(10, 4))
plot_acf(subset_df['sales'], lags=30)
plt.title('Autocorrelation Plot (Sales)')
plt.tight_layout()
plt.show()
```

```
<Figure size 1000x400 with 0 Axes>
```



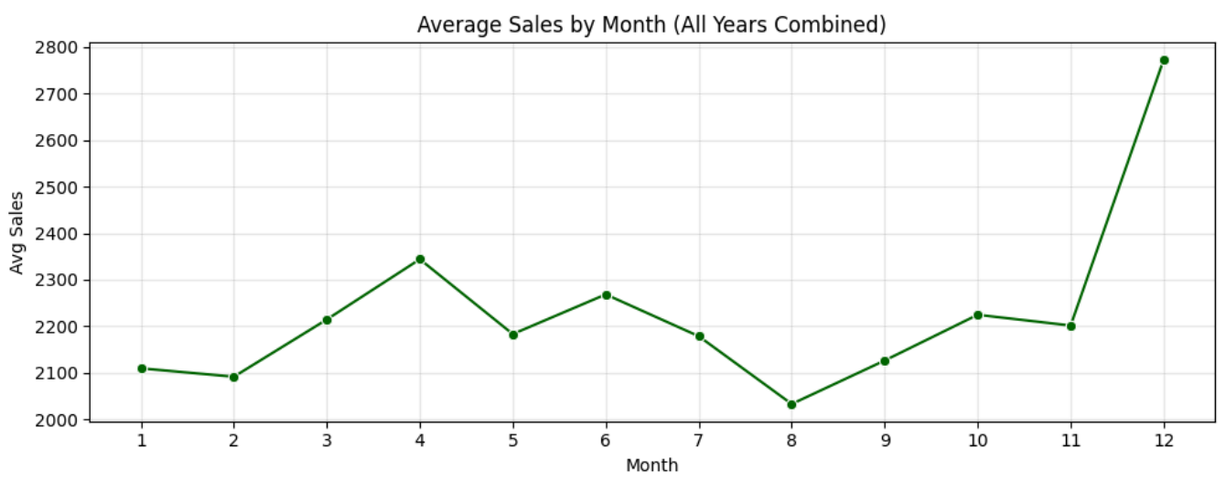
➕ 7. Sales by Year

```
# Extract year from date
subset_df['year'] = subset_df.index.year

# Group and plot
yearly_avg = subset_df.groupby('year')['sales'].mean()

plt.figure(figsize=(8, 4))
sns.barplot(x=yearly_avg.index.astype(str), y=yearly_avg.values, palette='viridis')
plt.title('Average Sales Per Year')
plt.xlabel('Year')
plt.ylabel('Avg Sales')
plt.grid(True, axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



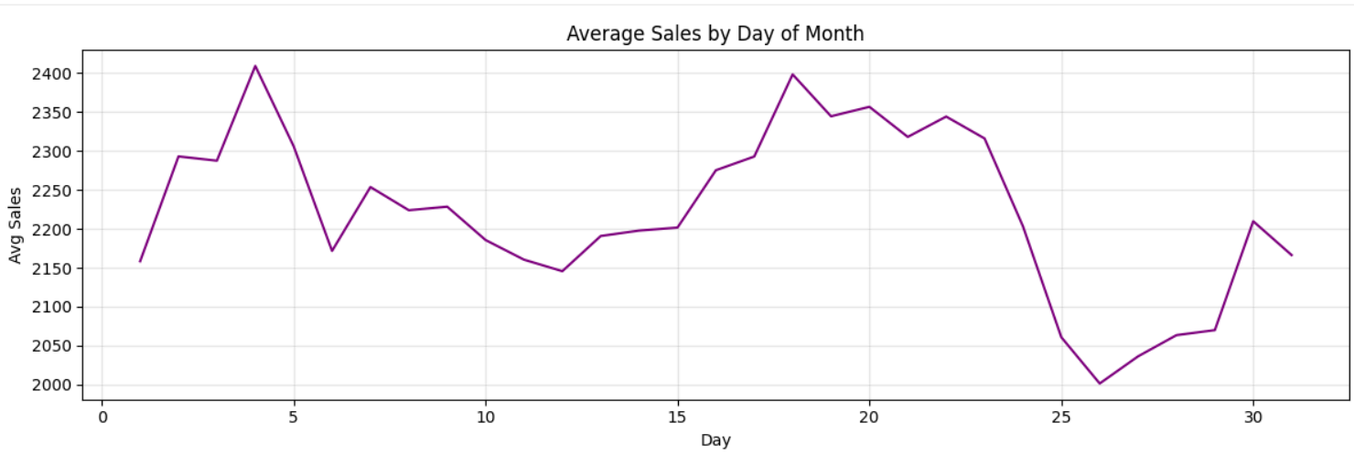
8. Sales by Month (Across All Years)

```
subset_df['month'] = subset_df.index.month

monthly_avg = subset_df.groupby('month')['sales'].mean()

plt.figure(figsize=(10, 4))
sns.lineplot(x=monthly_avg.index, y=monthly_avg.values, marker='o', color='darkgreen')
plt.title('Average Sales by Month (All Years Combined)')
plt.xlabel('Month')
plt.ylabel('Avg Sales')
plt.xticks(range(1, 13))
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



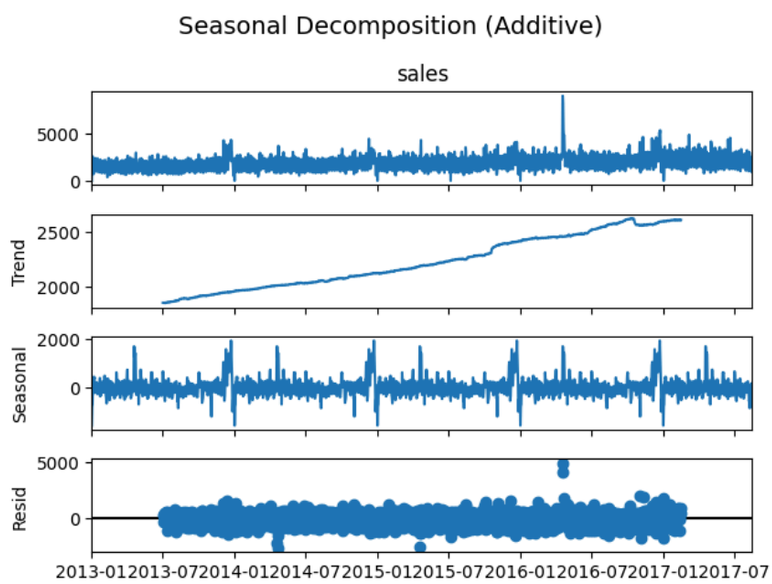
9. Sales by Day of Month

```
subset_df['day'] = subset_df.index.day

day_avg = subset_df.groupby('day')['sales'].mean()

plt.figure(figsize=(12, 4))
sns.lineplot(x=day_avg.index, y=day_avg.values, color='purple')
plt.title('Average Sales by Day of Month')
plt.xlabel('Day')
plt.ylabel('Avg Sales')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



10. Seasonal Decomposition Plot

```
from statsmodels.tsa.seasonal import seasonal_decompose

# Ensure daily frequency
ts = subset_df['sales'].asfreq('D')

# Fill missing values
ts = ts.fillna(method='ffill')

# Decompose
decomp = seasonal_decompose(ts, model='additive', period=365)

# Plot
decomp.plot()
plt.suptitle('Seasonal Decomposition (Additive)', fontsize=14)
plt.tight_layout()
plt.show()

missing_days = ts[ts.isna()]
print(f"Missing days count: {missing_days.shape[0]}")
```




```
Missing days count: 0
```

🧠 Insights

- Sales have clear weekly and monthly patterns.
- Holidays appear to affect demand slightly, with higher variation on event days.
- Promotions positively influence sales, as shown in the boxplot.
- Autocorrelation confirms short-term time dependencies, especially weekly lag effects.
- Year-wise sales have remained relatively stable with minor seasonal peaks.
- Month-wise trends show recurring demand spikes in specific months.
- Day-of-month analysis shows no strong fixed-day effect, but slight dips in mid-month.
- Seasonal decomposition separates trend, seasonality, and residuals — confirming our earlier findings.

🔹 Time Series Preparation – Train/Test Split & Stationarity Check

Before training time series models, it's essential to prepare the data:

- ✂️ Split data into training and testing sets
- 🔁 Apply log transformation to stabilize variance
- ✅ Check for stationarity using Augmented Dickey-Fuller (ADF) test
- 📈 Visualize rolling mean & std to confirm stability

✅ Train-Test Split

```
# Reset index if needed
data = subset_df[['sales']].copy()
data = data.asfreq('D')  # ensure daily frequency
data = data.fillna(method='ffill')

# Set cutoff date (e.g., last 3 months for testing)
split_date = '2017-07-01'

train = data.loc[:split_date].copy()
test = data.loc[split_date:].copy()

print(f"Train shape: {train.shape}")
print(f"Test shape: {test.shape}")
```

```
Train shape: (1643, 1)
Test shape: (46, 1)
```

✅ Log Transformation (Optional but useful)

```
train['log_sales'] = np.log1p(train['sales'])  # log(1 + x)
test['log_sales'] = np.log1p(test['sales'])
```

✅ ADF Test – Augmented Dickey-Fuller

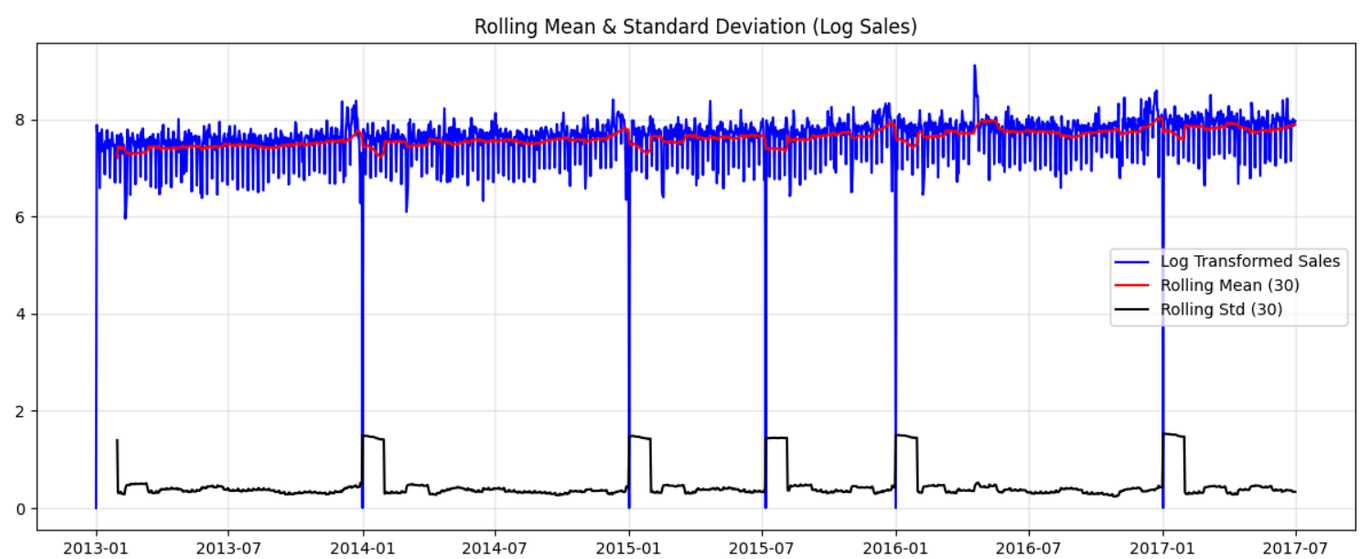
```
from statsmodels.tsa.stattools import adfuller

result = adfuller(train['log_sales'].dropna())
print("ADF Statistic:", result[0])
print("p-value:", result[1])
for key, value in result[4].items():
    print(f"Critical Value ({key}): {value}")

if result[1] < 0.05:
    print("✅ The series is stationary.")
else:
    print("⚠️ The series is non-stationary.")
```

```
ADF Statistic: -6.1389920357466785
p-value: 8.056591705344745e-08
Critical Value (1%): -3.4343905216323187
Critical Value (5%): -2.8633246558950796
Critical Value (10%): -2.567720112819826
✅ The series is stationary.
```

- **The ADF test shows whether the series is stationary.**
- **A p-value below 0.05 confirms that the series is already stationary.**

✅ Rolling Mean & Standard Deviation Plot

```
rol_mean = train['log_sales'].rolling(window=30).mean()
rol_std = train['log_sales'].rolling(window=30).std()

plt.figure(figsize=(12, 5))
plt.plot(train['log_sales'], color='blue', label='Log Transformed Sales')
plt.plot(rol_mean, color='red', label='Rolling Mean (30)')
plt.plot(rol_std, color='black', label='Rolling Std (30)')
plt.legend(loc='best')
plt.title('Rolling Mean & Standard Deviation (Log Sales)')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



- **Rolling plots help visually confirm if the mean and variance are stable over time.**
- **If the series is non-stationary, differencing or decomposition may be required.**

🔹 Model Training & Forecasting (ARIMA, Prophet, XGBoost)

Now that the data is clean and prepared, we build forecasting models:

- 🌀 ARIMA for linear time-based forecasting
- 🔮 Prophet for handling seasonality and holidays
- 🚀 XGBoost as a machine learning-based time series model

We'll train, forecast, and compare performance using RMSE, MAE, and MAPE.

✅ ARIMA Modeling

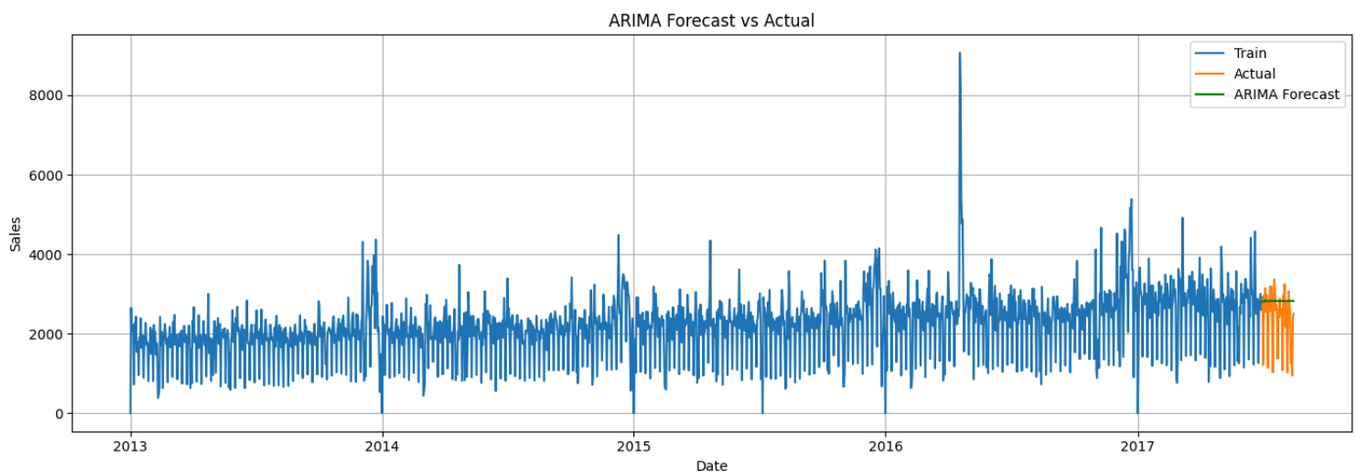
```
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_error

# Fit ARIMA on log-transformed data
model_arima = ARIMA(train['log_sales'], order=(5,1,0))
model_arima_fit = model_arima.fit()

# Forecast steps ahead
forecast_steps = len(test)
forecast_log = model_arima_fit.forecast(steps=forecast_steps)
forecast_arima = np.expm1(forecast_log)  # convert back from log
```

```
# Evaluation
rmse_arima = np.sqrt(mean_squared_error(test['sales'], forecast_arima))
mae_arima = mean_absolute_error(test['sales'], forecast_arima)

print("ARIMA RMSE:", rmse_arima)
print("ARIMA MAE:", mae_arima)
```

```
ARIMA RMSE: 779.4864434666279
ARIMA MAE: 528.7938912408387
```

📈 Plot ARIMA Forecast

```
plt.figure(figsize=(14, 5))
plt.plot(train.index, train['sales'], label='Train')
plt.plot(test.index, test['sales'], label='Actual')
plt.plot(test.index, forecast_arima, label='ARIMA Forecast', color='green')
plt.title('ARIMA Forecast vs Actual')
plt.xlabel('Date')
plt.ylabel('Sales')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



✅ Prepare Data for Prophet

```
from prophet import Prophet

# Prophet needs 'ds' and 'y'
prophet_df = subset_df.reset_index()[['date', 'sales']].rename(columns={'date': 'ds', 'sales': 'y'})

# Filter again for same subset
prophet_df = prophet_df[(prophet_df['ds'] < '2017-08-01') & (prophet_df['ds'] >= '2013-01-01')]

# Split
train_prophet = prophet_df[prophet_df['ds'] < '2017-07-01']
test_prophet = prophet_df[prophet_df['ds'] >= '2017-07-01']
```

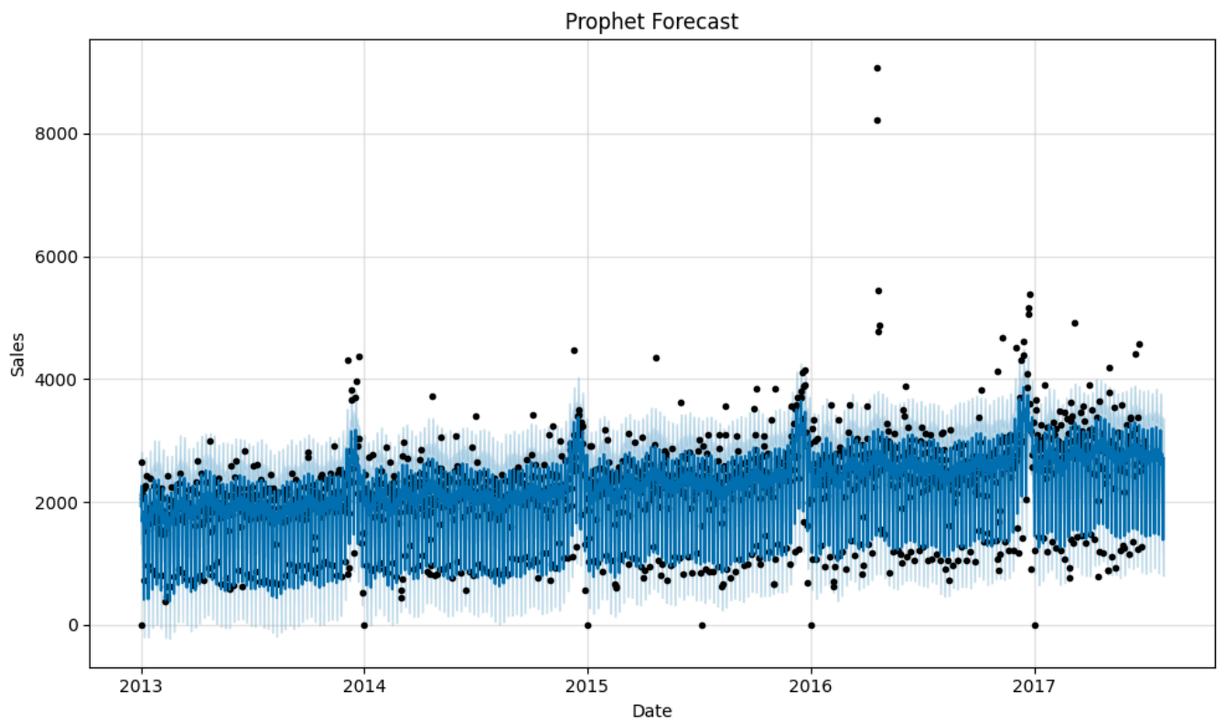
✅ Fit & Forecast Prophet

```
m = Prophet()
m.fit(train_prophet)

future = m.make_future_dataframe(periods=len(test_prophet))
forecast = m.predict(future)

# Evaluation
y_true = test_prophet['y'].values
y_pred = forecast.iloc[-len(test_prophet):]['yhat'].values

rmse_prophet = np.sqrt(mean_squared_error(y_true, y_pred))
mae_prophet = mean_absolute_error(y_true, y_pred)

print("Prophet RMSE:", rmse_prophet)
print("Prophet MAE:", mae_prophet)
```

```
INFO:prophet:Disabling daily seasonality. Run prophet with daily_seasonality=True to override this.
DEBUG:cmdstanpy:input tempfile: /tmp/tmp9j18x7sm/wzv9gh1m.json
DEBUG:cmdstanpy:input tempfile: /tmp/tmp9j18x7sm/w0alvu2f.json
DEBUG:cmdstanpy:idx 0
DEBUG:cmdstanpy:running CmdStan, num_threads: None
DEBUG:cmdstanpy:CmdStan args: ['/usr/local/lib/python3.12/dist-packages/prophet/stan_model/prophet_model.bin', 'random', 'seed=39320', 'data',
13:29:25 - cmdstanpy - INFO - Chain [1] start processing
INFO:cmdstanpy:Chain [1] start processing
13:29:25 - cmdstanpy - INFO - Chain [1] done processing
INFO:cmdstanpy:Chain [1] done processing
Prophet RMSE: 236.37296236734934
Prophet MAE: 196.4841414748084
```

📈 Plot Prophet Forecast

```
fig = m.plot(forecast)
plt.title('Prophet Forecast')
plt.xlabel('Date')
plt.ylabel('Sales')
plt.tight_layout()
plt.show()
```



🔹 Forecasting with XGBoost (Lag-Based ML Approach)

To leverage machine learning for time series, we engineer features from past values and time components.

We'll use:

- Lag features (t-1, t-7, etc.)
- Rolling means
- Date-based components
- Then train an XGBoost model to forecast future sales

⚙️ Feature Engineering

```
df_ml = subset_df[['sales']].copy()
df_ml['day'] = df_ml.index.day
df_ml['month'] = df_ml.index.month
df_ml['weekday'] = df_ml.index.weekday
df_ml['year'] = df_ml.index.year

# Lag features
df_ml['lag_1'] = df_ml['sales'].shift(1)
df_ml['lag_7'] = df_ml['sales'].shift(7)
df_ml['rolling_mean_7'] = df_ml['sales'].shift(1).rolling(window=7).mean()

# Drop NaNs
df_ml.dropna(inplace=True)
```

🎗️ Train-Test Split

```
X = df_ml.drop('sales', axis=1)
y = df_ml['sales']

# Split index (same as previous)
split_date = '2017-07-01'
X_train = X.loc[:split_date]
y_train = y.loc[:split_date]
X_test = X.loc[split_date:]
y_test = y.loc[split_date:]
```

🚀 Train XGBoost Model

```
from xgboost import XGBRegressor

model_xgb = XGBRegressor(n_estimators=100, learning_rate=0.1, random_state=42)
model_xgb.fit(X_train, y_train)

# Predict
y_pred_xgb = model_xgb.predict(X_test)
```

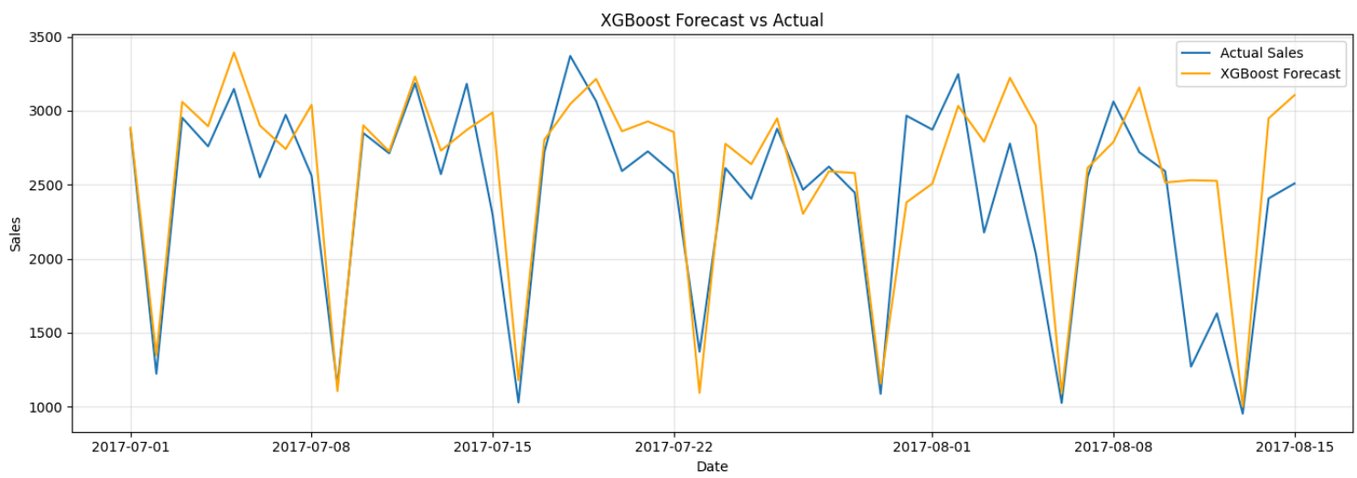
📊 Evaluation

```
from sklearn.metrics import mean_squared_error, mean_absolute_error

rmse_xgb = np.sqrt(mean_squared_error(y_test, y_pred_xgb))
mae_xgb = mean_absolute_error(y_test, y_pred_xgb)

print("XGBoost RMSE:", rmse_xgb)
print("XGBoost MAE:", mae_xgb)
```

```
XGBoost RMSE: 385.85649581818564
XGBoost MAE: 281.407518469769
```

📈 Plot Actual vs Predicted

```
plt.figure(figsize=(14, 5))
plt.plot(y_test.index, y_test, label='Actual Sales')
plt.plot(y_test.index, y_pred_xgb, label='XGBoost Forecast', color='orange')
plt.title('XGBoost Forecast vs Actual')
plt.xlabel('Date')
plt.ylabel('Sales')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



XGBoost performed well by learning patterns from lag and rolling statistics.

Key Pros:

- Learns non-linear patterns
- Can include external regressors
- Fast to train

We'll now compare performance across all models.

🔹 Model Comparison – ARIMA vs Prophet vs XGBoost

After training all three models, we now compare their performance using:

- 🔢 RMSE (Root Mean Squared Error)
- 🔢 MAE (Mean Absolute Error)
- 📉 MAPE (Mean Absolute Percentage Error)

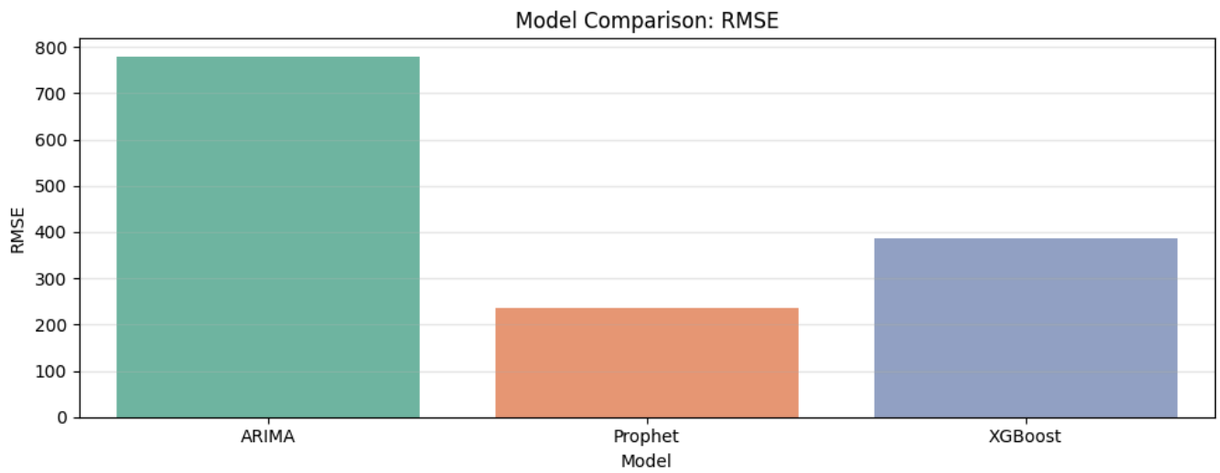
✅ 📊 Comparison Table

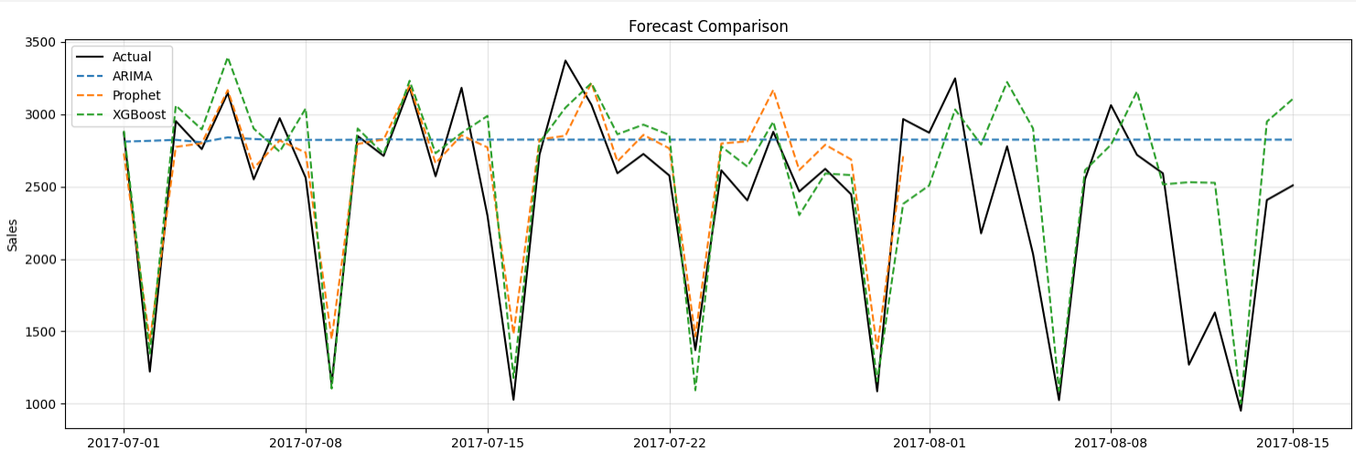
```
from sklearn.metrics import mean_absolute_percentage_error

# Calculate MAPE for Prophet and XGBoost
mape_arima = mean_absolute_percentage_error(test['sales'], forecast_arima)
mape_prophet = mean_absolute_percentage_error(y_true, y_pred)
mape_xgb = mean_absolute_percentage_error(y_test, y_pred_xgb)

# Final comparison
results = pd.DataFrame({
    'Model': ['ARIMA', 'Prophet', 'XGBoost'],
    'RMSE': [rmse_arima, rmse_prophet, rmse_xgb],
    'MAE': [mae_arima, mae_prophet, mae_xgb],
    'MAPE': [mape_arima, mape_prophet, mape_xgb]
})

results.sort_values('RMSE')
```

	Model	RMSE	MAE	MAPE
1	Prophet	236.372962	196.484141	0.095497
2	XGBoost	385.856496	281.407518	0.132098
0	ARIMA	779.486443	528.793891	0.360249

📊 Visual Comparison – Barplot

```
plt.figure(figsize=(10, 4))
sns.barplot(x='Model', y='RMSE', data=results, palette='Set2')
plt.title('Model Comparison: RMSE')
plt.ylabel('RMSE')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



✅ All 3 Predictions in One Plot

```
plt.figure(figsize=(15, 5))
plt.plot(test.index, test['sales'], label='Actual', color='black')
plt.plot(test.index, forecast_arima, label='ARIMA', linestyle='--')
plt.plot(test_prophet['ds'], y_pred, label='Prophet', linestyle='--')
plt.plot(y_test.index, y_pred_xgb, label='XGBoost', linestyle='--')
plt.title('Forecast Comparison')
plt.ylabel('Sales')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```